

Lecture 18: Summarizing Data

Heather Mattie, PhD

October 30, 2025

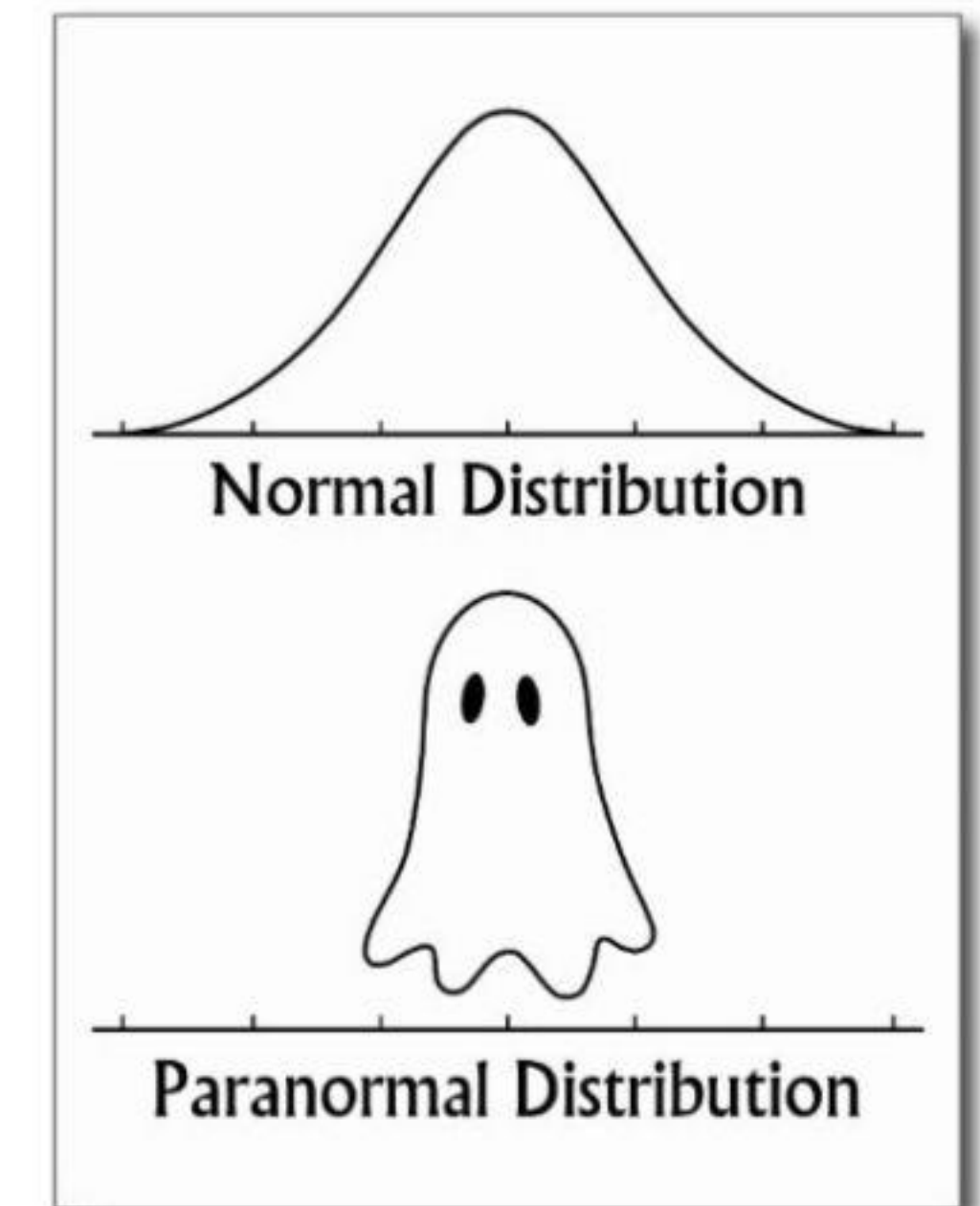
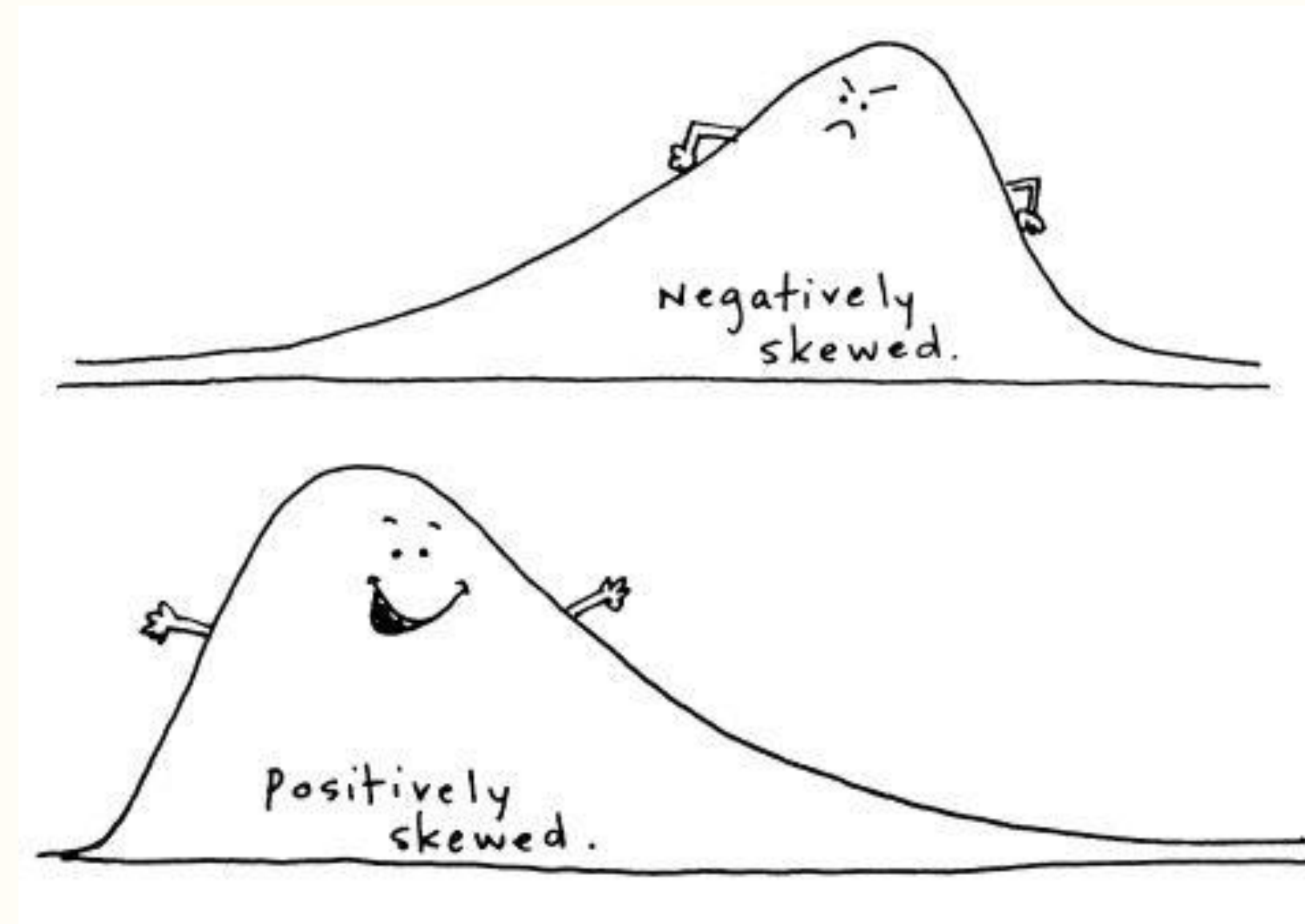
Recipe of the Day!

Dia de los Muertos recipes



Announcements

- No lab this week
- Midterm due **Friday, November 7**
 - No late days may be used



Coding question of the day

Using the **gapminder** dataset, create a new data frame with only the **country**, **year**, **life_expectancy**, and **fertility** columns. Use the **pivot_longer()** function to reshape the dataset so that the **life_expectancy** and **fertility** columns are combined into a single value column called "**value**". The column names should be stored in a new column called "**health_indicator**".

Be sure to run this code first

```
# Load necessary libraries
library(dslabs)
library(tidyr)
library(dplyr)

# Load the gapminder dataset
data("gapminder")
```

Summarizing data

Summarizing Data Roadmap

1. Numerical summaries
2. Frequency tables
3. The **tableone** package
4. Reporting missing data

Summarizing data

Numerical summaries

Heather Mattie, PhD

Numerical summaries

Numerical summaries in statistics are **concise** ways to describe the key characteristics of a dataset using numbers

Measures of **centrality**

- Mean, median

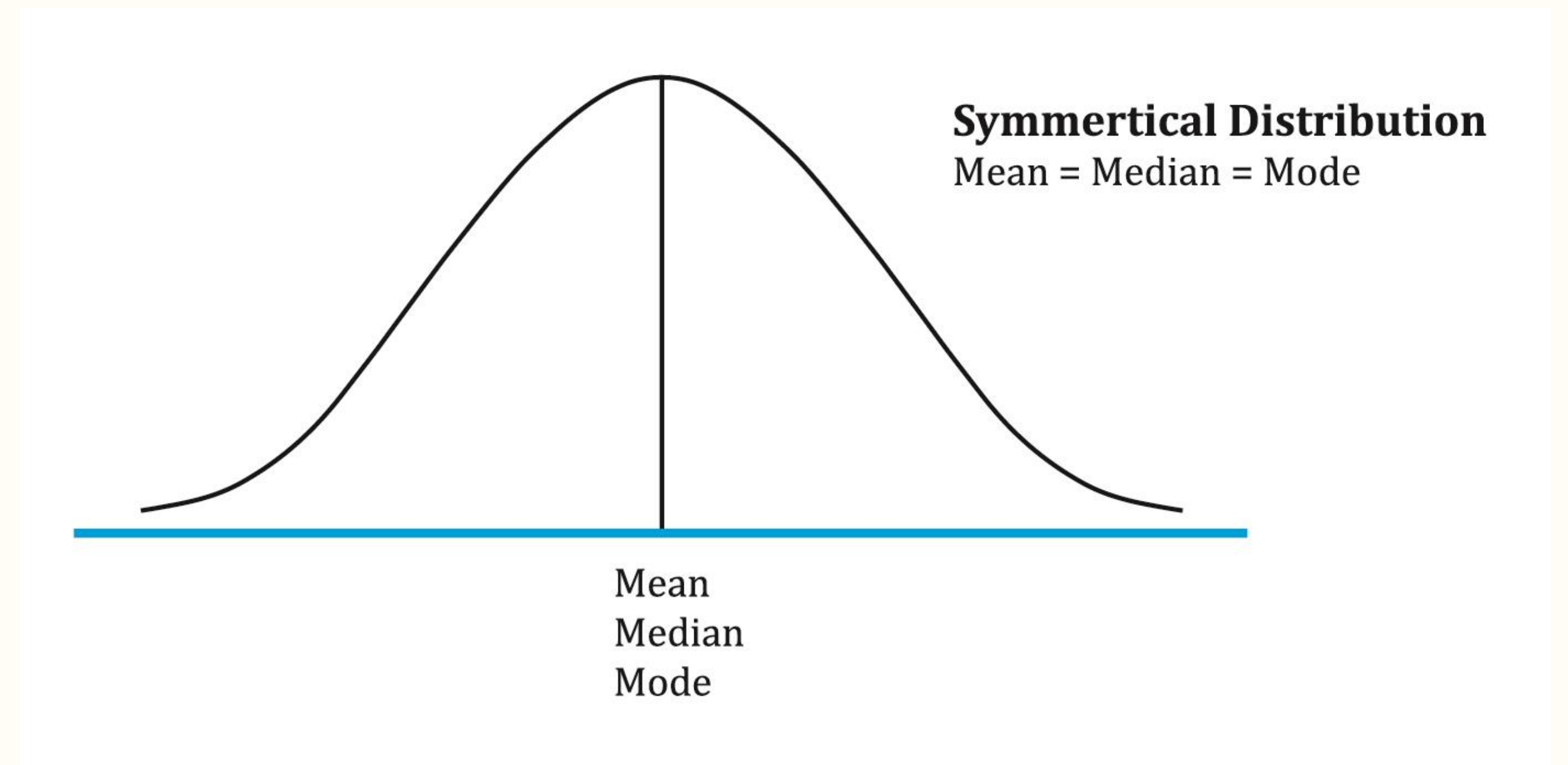
Measures of **dispersion**

- Standard deviation, variance, range, interquartile range

Mean

The **mean** is the average of all values in a dataset, calculated by summing all values and dividing by the total number of values.

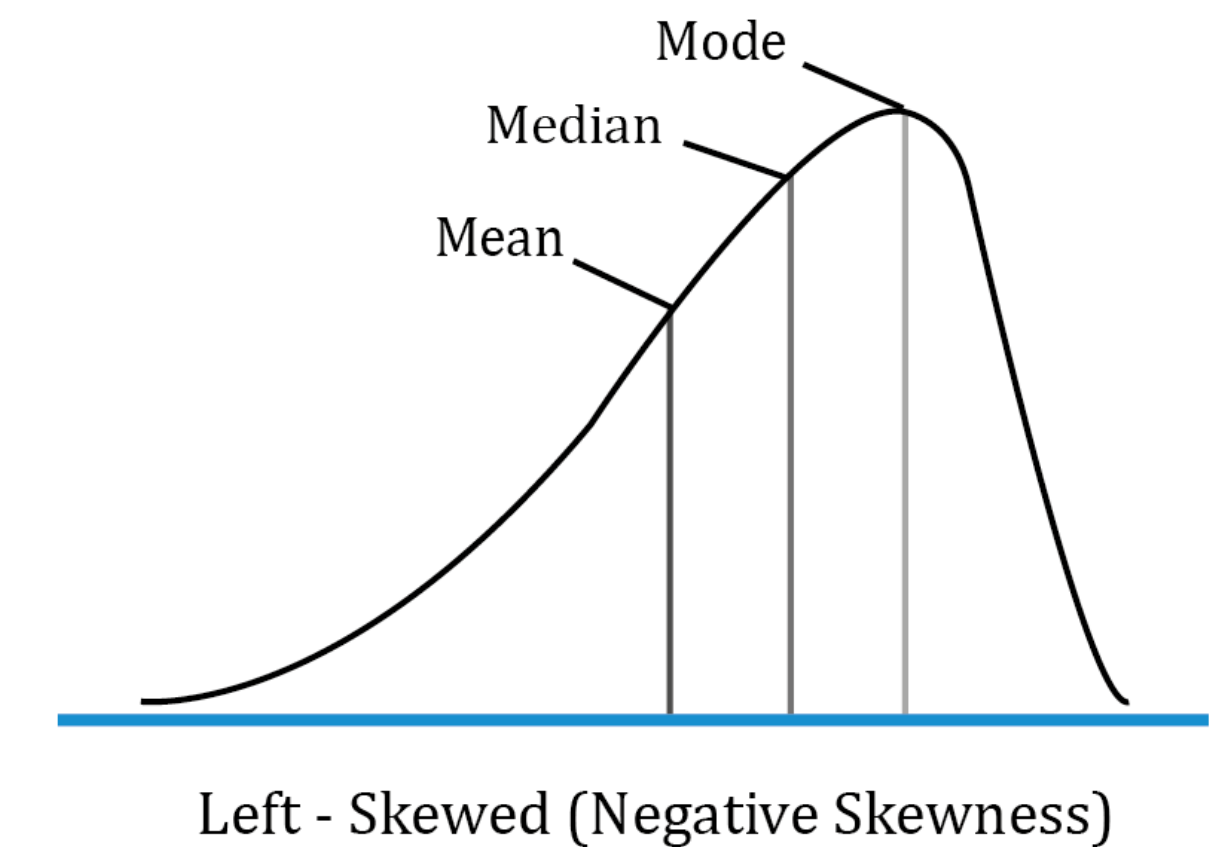
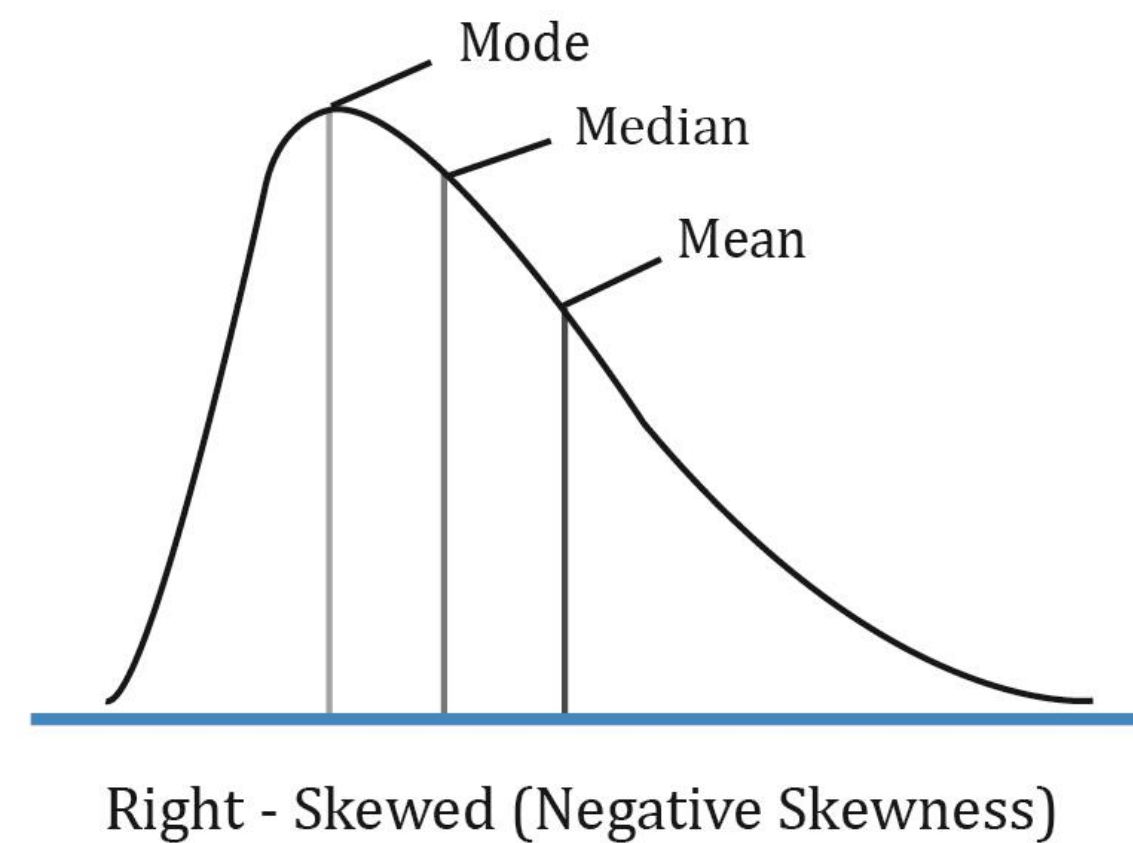
- The mean is the measure of centrality reported for **symmetric** distributions of data



Median

The **median** is the middle value in a sorted dataset. It divides the data into two halves, with 50% of the values below it and 50% above it.

- The median is the measure of centrality reported for **skewed** distributions of data

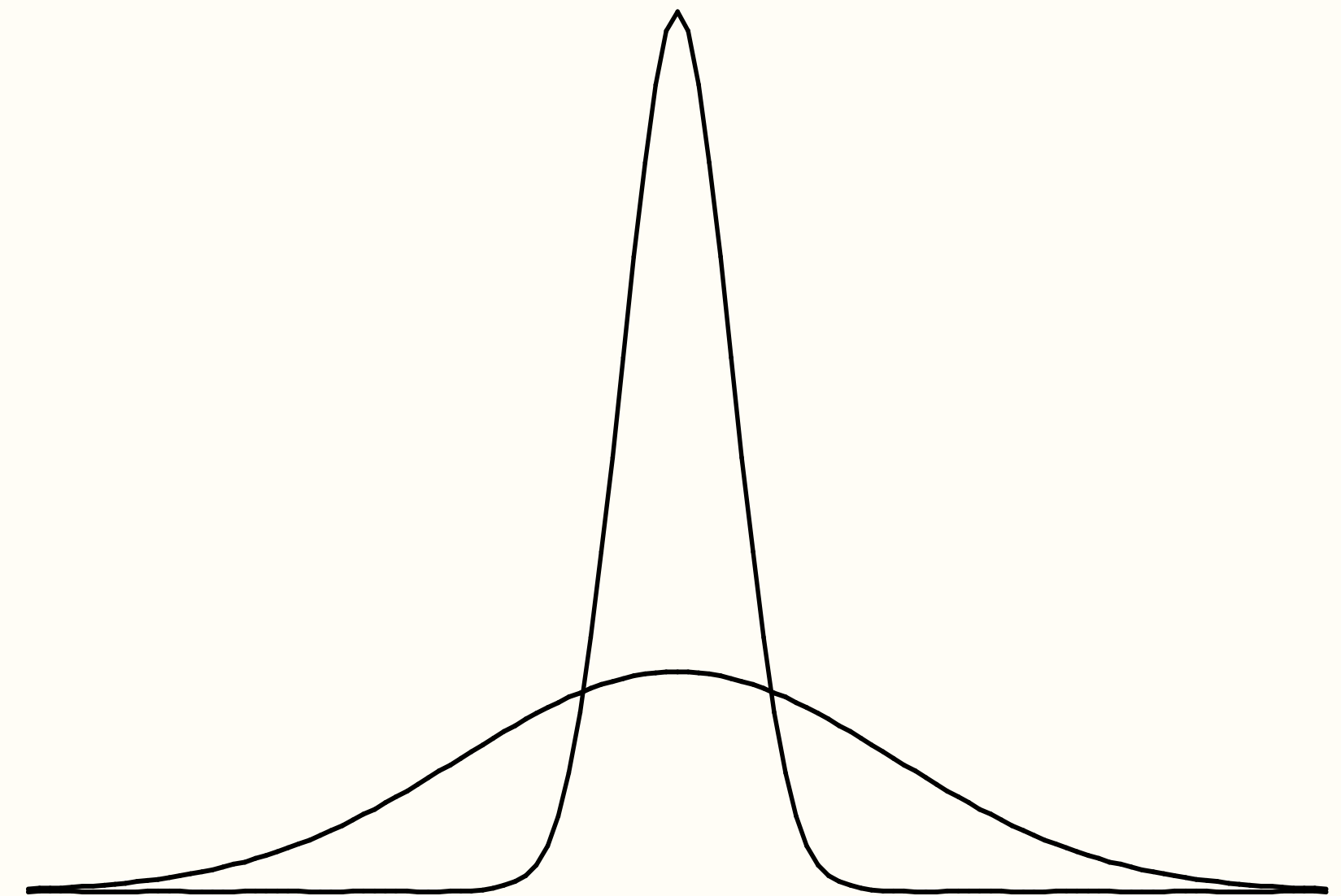


Standard deviation and variance

Standard deviation is a measure of the spread of data around the mean. A higher standard deviation indicates greater variability.

- Standard deviation is the measure of dispersion reported for **symmetric** distributions of data

Variance is the square of standard deviation.

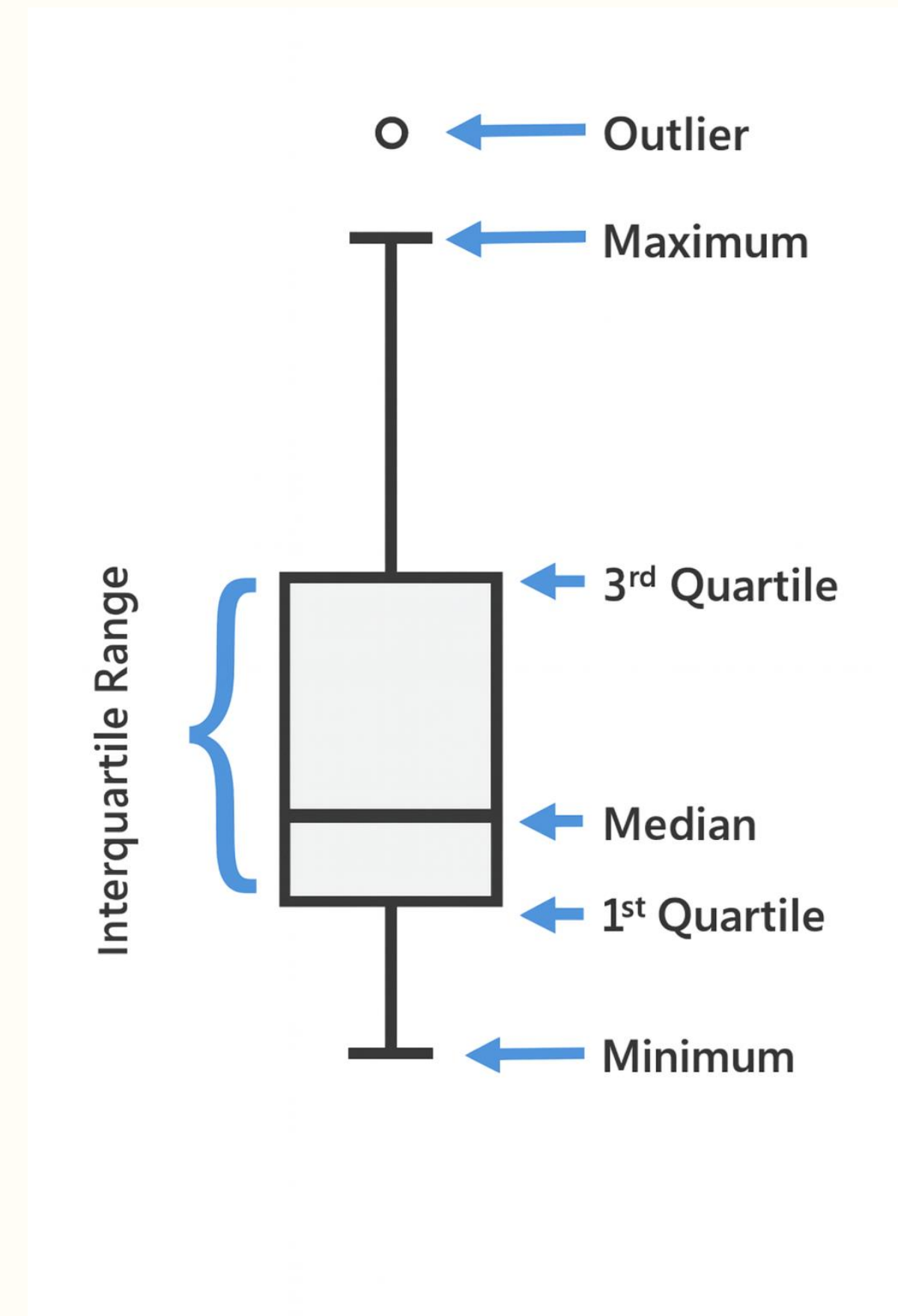


Range and interquartile range (IQR)

The **range** is the difference between the largest and smallest values in a dataset.

The **interquartile range (IQR)** is the difference between the third quartile (75th percentile) and the first quartile (25th percentile), representing the spread of the middle 50% of the data.

- The range or IQR is the measure of spread reported for skewed distributions



Numerical summaries in R

The **summary** function will return the minimum value, first quartile, median, mean, third quartile, and maximum value

```
library(dslabs)
library(tidyverse)
data(gapminder)
summary(gapminder)
```

```
##           country      year infant_mortality life_expectancy
## Albania      : 57   Min.   :1960   Min.    : 1.50   Min.    :13.20
## Algeria      : 57   1st Qu.:1974   1st Qu.: 16.00   1st Qu.:57.50
## Angola       : 57   Median :1988   Median : 41.50   Median :67.54
## Antigua and Barbuda: 57   Mean    :1988   Mean    : 55.31   Mean    :64.81
## Argentina    : 57   3rd Qu.:2002   3rd Qu.: 85.10   3rd Qu.:73.00
## Armenia      : 57   Max.    :2016   Max.    :276.90   Max.    :83.90
## (Other)      :10203   NA's    :1453
## fertility    population      gdp      continent
## Min.   :0.840   Min.   :3.124e+04   Min.   :4.040e+07   Africa :2907
## 1st Qu.:2.200   1st Qu.:1.333e+06   1st Qu.:1.846e+09   Americas:2052
## Median :3.750   Median :5.009e+06   Median :7.794e+09   Asia   :2679
## Mean   :4.084   Mean   :2.701e+07   Mean   :1.480e+11   Europe :2223
## 3rd Qu.:6.000   3rd Qu.:1.523e+07   3rd Qu.:5.540e+10   Oceania : 684
## Max.   :9.220   Max.   :1.376e+09   Max.   :1.174e+13
## NA's   :187    NA's   :185        NA's   :2972
##           region
## Western Asia :1026
## Eastern Africa : 912
## Western Africa : 912
## Caribbean     : 741
## South America : 684
## Southern Europe: 684
## (Other)       :5586
```

Numerical summaries in R

The **summary** function will return the minimum value, first quartile, median, mean, third quartile, and maximum value

```
gapminder %>%  
  filter(year == 1986) %>%  
  select(life_expectancy) %>%  
  summary()
```

```
## life_expectancy  
## Min.      :41.39  
## 1st Qu.:58.96  
## Median :68.48  
## Mean    :65.57  
## 3rd Qu.:71.85  
## Max.     :78.38
```

Numerical summaries in R

The **mean** function returns the mean

The **median** function returns the median

```
gapminder_1986 <- gapminder %>%  
  filter(year == 1986 & !is.na(life_expectancy))  
  
le_1986 <- gapminder_1986$life_expectancy  
  
mean(le_1986)
```

```
## [1] 65.57362
```

```
median(le_1986)
```

```
## [1] 68.48
```

Numerical summaries in R

The **sd** function returns the standard deviation

The **var** function returns the variance

The **range** function returns the minimum and maximum values

The **IQR** function returns the interquartile range

```
sd(le_1986)
```

```
## [1] 8.679824
```

```
var(le_1986)
```

```
## [1] 75.33935
```

```
range(le_1986)
```

```
## [1] 41.39 78.38
```

```
IQR(le_1986)
```

```
## [1] 12.89
```

Numerical summaries in R

The **group_by** and **summarize** functions return summary values for different groups

- Always **ungroup()** when finished using **group_by**

```
gapminder %>%
  filter(year == 1986 & !is.na(life_expectancy)) %>%
  group_by(continent) %>%
  summarize(le_mean = mean(life_expectancy),
            le_median = median(life_expectancy),
            le_sd = sd(life_expectancy),
            le_var = var(life_expectancy),
            le_range = range(life_expectancy),
            le_IQR = IQR(life_expectancy)) %>%
  ungroup()
```

```
## # A tibble: 10 × 7
##   continent le_mean le_median le_sd le_var le_range le_IQR
##   <fct>      <dbl>    <dbl> <dbl> <dbl>    <dbl>    <dbl>
## 1 Africa      56.0      55.5  7.03  49.5     41.4     9.82
## 2 Africa      56.0      55.5  7.03  49.5     71.6     9.82
## 3 Americas    69.4      69.8  4.97  24.7     52.8     4.78
## 4 Americas    69.4      69.8  4.97  24.7     76.5     4.78
## 5 Asia        66.5      68.3  6.84  46.8     49.0     9.40
## 6 Asia        66.5      68.3  6.84  46.8     78.2     9.40
## 7 Europe      73.5      74.1  2.63   6.93     67.3     4.57
## 8 Europe      73.5      74.1  2.63   6.93     78.4     4.57
## 9 Oceania     65.2      65.4  6.52  42.6     55.2     8.54
## 10 Oceania    65.2      65.4  6.52  42.6     76.1     8.54
```

Numerical summaries in R

The **group_by** and **reframe** functions return summary values for different groups

- Do not need to include **ungroup()** when using the **reframe** function

```
gapminder %>%  
  filter(year == 1986 & !is.na(life_expectancy)) %>%  
  group_by(continent) %>%  
  reframe(le_mean = mean(life_expectancy),  
          le_median = median(life_expectancy),  
          le_sd = sd(life_expectancy),  
          le_var = var(life_expectancy),  
          le_range = range(life_expectancy),  
          le_IQR = IQR(life_expectancy)) #>%>%
```

```
## # A tibble: 10 × 7  
##   continent le_mean le_median le_sd le_var le_range le_IQR  
##   <fct>      <dbl>    <dbl> <dbl> <dbl>    <dbl> <dbl>  
## 1 Africa      56.0      55.5  7.03  49.5     41.4  9.82  
## 2 Africa      56.0      55.5  7.03  49.5     71.6  9.82  
## 3 Americas    69.4      69.8  4.97  24.7     52.8  4.78  
## 4 Americas    69.4      69.8  4.97  24.7     76.5  4.78  
## 5 Asia        66.5      68.3  6.84  46.8     49.0  9.40  
## 6 Asia        66.5      68.3  6.84  46.8     78.2  9.40  
## 7 Europe      73.5      74.1  2.63   6.93     67.3  4.57  
## 8 Europe      73.5      74.1  2.63   6.93     78.4  4.57  
## 9 Oceania     65.2      65.4  6.52  42.6     55.2  8.54  
## 10 Oceania    65.2      65.4  6.52  42.6     76.1  8.54
```

Summary

- Numerical summaries are concise ways to describe the key characteristics of a dataset using numbers
- The mean and standard deviation are reported when the data distribution is symmetric
- The median, range, and interquartile range are reported when the data distribution is skewed
- There are several numerical summary functions available in base R and the **tidyverse** package

Summarizing data

Frequency tables

Heather Mattie, PhD

Frequency tables

- **Frequency tables** are used to summarize nominal and ordinal measurements falling into categories
- They list each of the possible values that the data can assume (the categories or classes), along with the number of times each value occurs
- Categories must not overlap, and are usually (but not always) chosen to be of equal width

Frequency tables

The **table** function displays a table of each unique value and its corresponding count

```
library(dslabs)
library(tidyverse)
data(gapminder)

gapminder <- gapminder %>%
  filter(year == 2000 & !is.na(fertility)) %>%
  mutate(fertility_cat = round(fertility))

fertility_table <- table(gapminder$continent, gapminder$fertility_cat)
fertility_table
```

```
##
##           1  2  3  4  5  6  7  8
## Africa    0  3  6  6 10 17  8  1
## Americas  0 17 13  5  1  0  0  0
## Asia       6 14 12 11  2  1  1  0
## Europe    22 17  0  0  0  0  0  0
## Oceania   0  4  1  5  2  0  0  0
```

Frequency tables

The **table** function displays a table of each unique value and its corresponding count

- Using the **with** function to indicate which data frame to use can cut down on code

```
fertility_table <- with(gapminder, table(continent, fertility_cat))  
fertility_table
```

```
##           fertility_cat  
## continent  1  2  3  4  5  6  7  8  
##   Africa    0  3  6  6 10 17  8  1  
## Americas    0 17 13  5  1  0  0  0  
##   Asia      6 14 12 11  2  1  1  0  
##   Europe    22 17  0  0  0  0  0  0  
## Oceania     0  4  1  5  2  0  0  0
```

Relative frequency tables

- **Relative frequency** is the proportion or percent of the whole that falls into a given category
- Divide the number of subjects within a category by the total number in the table for a **proportion**; multiply the proportion by 100 for a **percent**
- Relative frequencies can be used to compare populations of different sizes

Relative frequency tables

The **prop.table** function returns a relative frequency table

```
fertility_prop_table <- prop.table(fertility_table)
fertility_prop_table
```

```
##           fertility_cat
## continent           1           2           3           4           5
## Africa  0.000000000 0.016216216 0.032432432 0.032432432 0.054054054
## Americas 0.000000000 0.091891892 0.070270270 0.027027027 0.005405405
## Asia     0.032432432 0.075675676 0.064864865 0.059459459 0.010810811
## Europe   0.118918919 0.091891892 0.000000000 0.000000000 0.000000000
## Oceania  0.000000000 0.021621622 0.005405405 0.027027027 0.010810811
##           fertility_cat
## continent           6           7           8
## Africa  0.091891892 0.043243243 0.005405405
## Americas 0.000000000 0.000000000 0.000000000
## Asia     0.005405405 0.005405405 0.000000000
## Europe   0.000000000 0.000000000 0.000000000
## Oceania  0.000000000 0.000000000 0.000000000
```

Relative frequency tables

The **prop.table** function returns a relative frequency table

- Using the **round** function to limit the number of decimal places provided in the table can make the output easier to read and interpret

```
fertility_prop_table <- round(prop.table(fertility_table), 2)
fertility_prop_table
```

```
##           fertility_cat
## continent      1      2      3      4      5      6      7      8
##   Africa  0.00 0.02 0.03 0.03 0.05 0.09 0.04 0.01
## Americas 0.00 0.09 0.07 0.03 0.01 0.00 0.00 0.00
##   Asia    0.03 0.08 0.06 0.06 0.01 0.01 0.01 0.00
##   Europe  0.12 0.09 0.00 0.00 0.00 0.00 0.00 0.00
## Oceania  0.00 0.02 0.01 0.03 0.01 0.00 0.00 0.00
```

Relative frequency tables

The **proportions** function also returns a relative frequency table

```
fertility_prop_table <- round(proportions(fertility_table), 2)
fertility_prop_table
```

```
##           fertility_cat
## continent      1      2      3      4      5      6      7      8
## Africa      0.00 0.02 0.03 0.03 0.05 0.09 0.04 0.01
## Americas    0.00 0.09 0.07 0.03 0.01 0.00 0.00 0.00
## Asia        0.03 0.08 0.06 0.06 0.01 0.01 0.01 0.00
## Europe      0.12 0.09 0.00 0.00 0.00 0.00 0.00 0.00
## Oceania     0.00 0.02 0.01 0.03 0.01 0.00 0.00 0.00
```

Relative frequency tables

The **proportions** function also returns a relative frequency table

- Multiplying by 100 returns percentages in the table

```
fertility_prop_table <- (round(proportions(fertility_table), 2) * 100)
fertility_prop_table
```

```
##           fertility_cat
## continent  1  2  3  4  5  6  7  8
##   Africa    0  2  3  3  5  9  4  1
##  Americas    0  9  7  3  1  0  0  0
##    Asia      3  8  6  6  1  1  1  0
##   Europe    12  9  0  0  0  0  0  0
##  Oceania     0  2  1  3  1  0  0  0
```

Summary

- Frequency tables summarize categorical data by displaying the number of times each value occurs
- Relative frequency tables summarize categorical data by displaying the proportion of values that fall into a given category
- The values in a relative frequency table can be transformed into percentages by multiplying each value by 100

Summarizing data

The **tableone** package

Heather Mattie, PhD

Organizing data summaries

- Oftentimes, it is useful to have numerical summaries, frequency table values, and relative frequency table values presented in one table
- This table is usually the first table in a scientific journal article and is referred to as “Table 1”

Table 1. Participant Demographics and Characteristics (N=504)

Characteristic	Total, <i>n</i> (%) N=504	Noticed calories, <i>n</i> (%) <i>n</i> =277 (55%)	Did not notice calories, <i>n</i> (%) <i>n</i> =227 (45%)	Chi-square or Wilcoxon, <i>p</i> -value
Participant age, in years				
18–25	117 (23.2%)	56 (20.2%)	61 (26.9%)	2.738, 0.0980
26–39	168 (33.3%)	96 (34.7%)	72 (31.7%)	0.362, 0.5476
≥40	219 (43.5%)	125 (45.1%)	94 (41.4%)	0.558, 0.4550
Participant race				
White	358 (71.0%)	207 (74.7%)	151 (66.5%)	3.697, 0.0545
Asian/Asian American	28 (5.6%)	15 (5.4%)	13 (5.7%)	<0.001, 0.9999
Black or African American	72 (14.3%)	33 (11.9%)	39 (17.2%)	2.413, 0.1203



tableone

The **tableone** package eases the creation of a “Table 1”

- Created in 2022 by a Harvard T.H. Chan alumnus
- Supports continuous and categorical variables
- Can provide p-values
- Supports stratification by groups

Package ‘tableone’

July 22, 2025

Type Package

Title Create 'Table 1' to Describe Baseline Characteristics with or without Propensity Score Weights

Version 0.13.2

Date 2022-04-15

Maintainer Kazuki Yoshida <kazukiyoshida@mail.harvard.edu>

Description Creates 'Table 1', i.e., description of baseline patient characteristics, which is essential in every medical research. Supports both continuous and categorical variables, as well as p-values and standardized mean differences. Weighted data are supported via the 'survey' package.

License GPL-2

Imports survey, MASS, e1071, zoo, gmodels, nlme, labelled

Suggests survival, testthat, Matrix, Matching, reshape2, ggplot2, knitr, geepack, lme4, lmerTest, rmarkdown

URL <https://github.com/kaz-yos/tableone>

tableone

```
library(dslabs)
library(tidyverse)
library(tableone)
data(gapminder)

table_df <- gapminder %>%
  filter(year == 1990)
table_df <- table_df[complete.cases(table_df),]

t1 <- CreateTableOne(vars = names(table_df), data = table_df)
print(t1)
```

	Overall	
n		164
country (%)		
Albania		1 (0.6)
Algeria		1 (0.6)
Angola		1 (0.6)
Antigua and Barbuda		1 (0.6)
Argentina		1 (0.6)
Armenia		1 (0.6)
Aruba		0 (0.0)
Australia		1 (0.6)
Austria		1 (0.6)
Azerbaijan		1 (0.6)
Bahamas		1 (0.6)
Bahrain		1 (0.6)
Bangladesh		1 (0.6)
Barbados		1 (0.6)



tableone

```
library(dslabs)
library(tidyverse)
library(tableone)
data(gapminder)

table_df <- gapminder %>%
  filter(year == 1990)
table_df <- table_df[complete.cases(table_df),]

t1 <- CreateTableOne(vars = names(table_df), data = table_df)
print(t1)
```

We don't need to print the
country, year, or region
variables

```
##      Zambia      1 ( 0.6)
##      Zimbabwe      1 ( 0.6)
##      year (mean (SD)) 1990.00 (0.00)
##      infant_mortality (mean (SD)) 49.70 (40.50)
##      life_expectancy (mean (SD)) 66.32 (8.56)
##      fertility (mean (SD)) 3.97 (1.89)
##      population (mean (SD)) 31297074.62 (115897913.10)
##      gdp (mean (SD)) 145422849909.40 (664869857190.42)
##      continent (%)
##      Africa      49 (29.9)
##      Americas     32 (19.5)
##      Asia         38 (23.2)
##      Europe       35 (21.3)
##      Oceania      10 ( 6.1)
##      region (%)
##      Australia and New Zealand      2 ( 1.2)
##      Caribbean      10 ( 6.1)
##      Central America      8 ( 4.9)
##      Central Asia      5 ( 3.0)
##      Eastern Africa     15 ( 9.1)
##      Eastern Asia      4 ( 2.4)
##      Eastern Europe     10 ( 6.1)
##      Melanesia      4 ( 2.4)
##      Micronesia      2 ( 1.2)
##      Middle Africa      8 ( 4.9)
##      Northern Africa      5 ( 3.0)
##      Northern America      2 ( 1.2)
```

tableone

```
vars <- c("infant_mortality", "life_expectancy",  
          "fertility", "population", "gdp",  
          "continent")  
  
t1 <- CreateTableOne(vars = vars, data = table_df)  
options(digits = 2)  
kableone(t1)
```

	Overall
n	164
infant_mortality (mean (SD))	49.70 (40.50)
life_expectancy (mean (SD))	66.32 (8.56)
fertility (mean (SD))	3.97 (1.89)
population (mean (SD))	31297074.62 (115897913.10)
gdp (mean (SD))	145422849909.40 (664869857190.42)
continent (%)	
Africa	49 (29.9)
Americas	32 (19.5)
Asia	38 (23.2)
Europe	35 (21.3)
Oceania	10 (6.1)



Variable distributions

The default for the **CreateTableOne** function is to report the **mean** and **standard deviation** for each continuous (numeric) variable

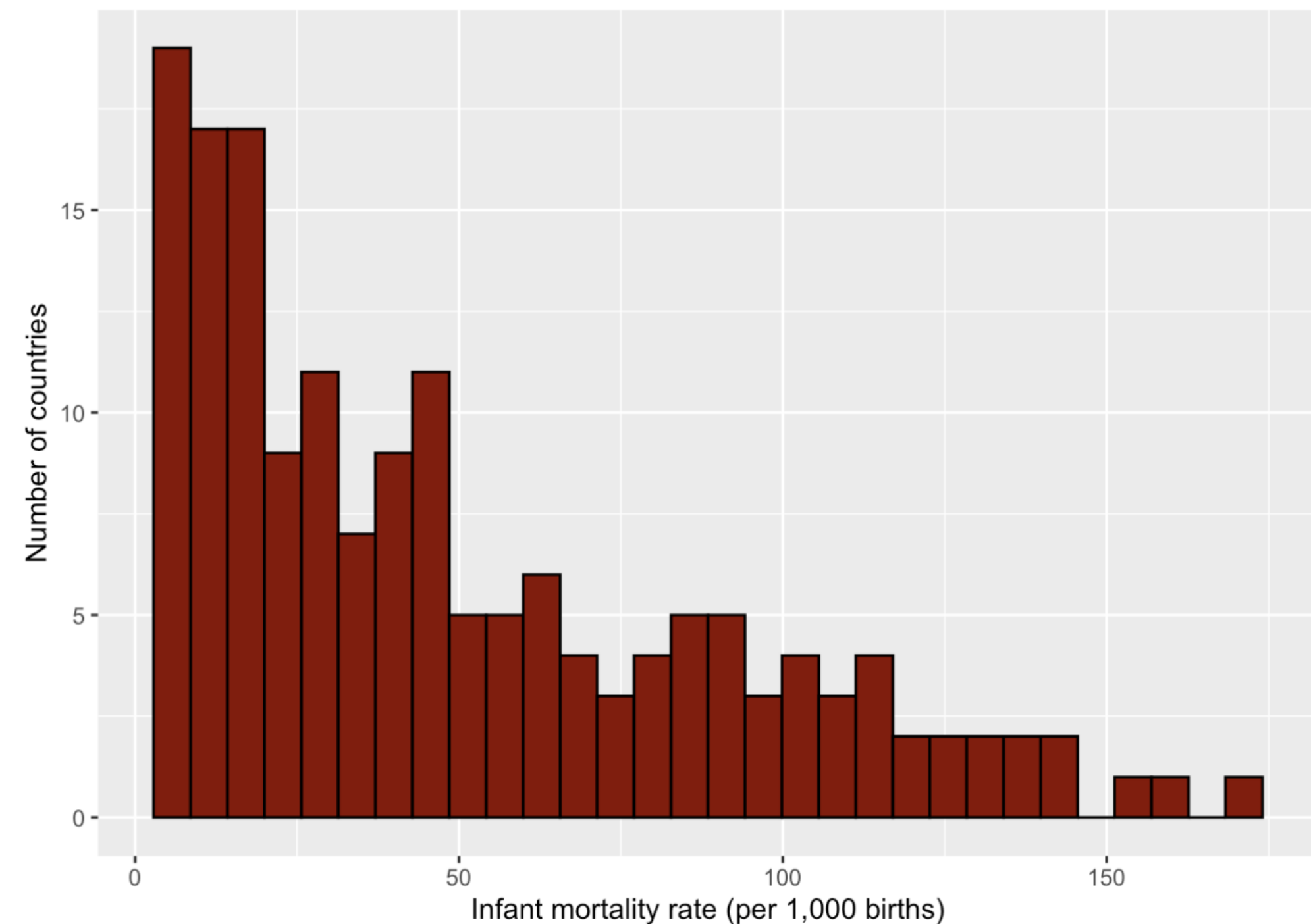
- Need to check that the distribution of each continuous variable is normally distributed – if not, need to report the **median** and **IQR**

Variable distributions

The default for the **CreateTableOne** function is to report the **mean** and **standard deviation** for each continuous (numeric) variable

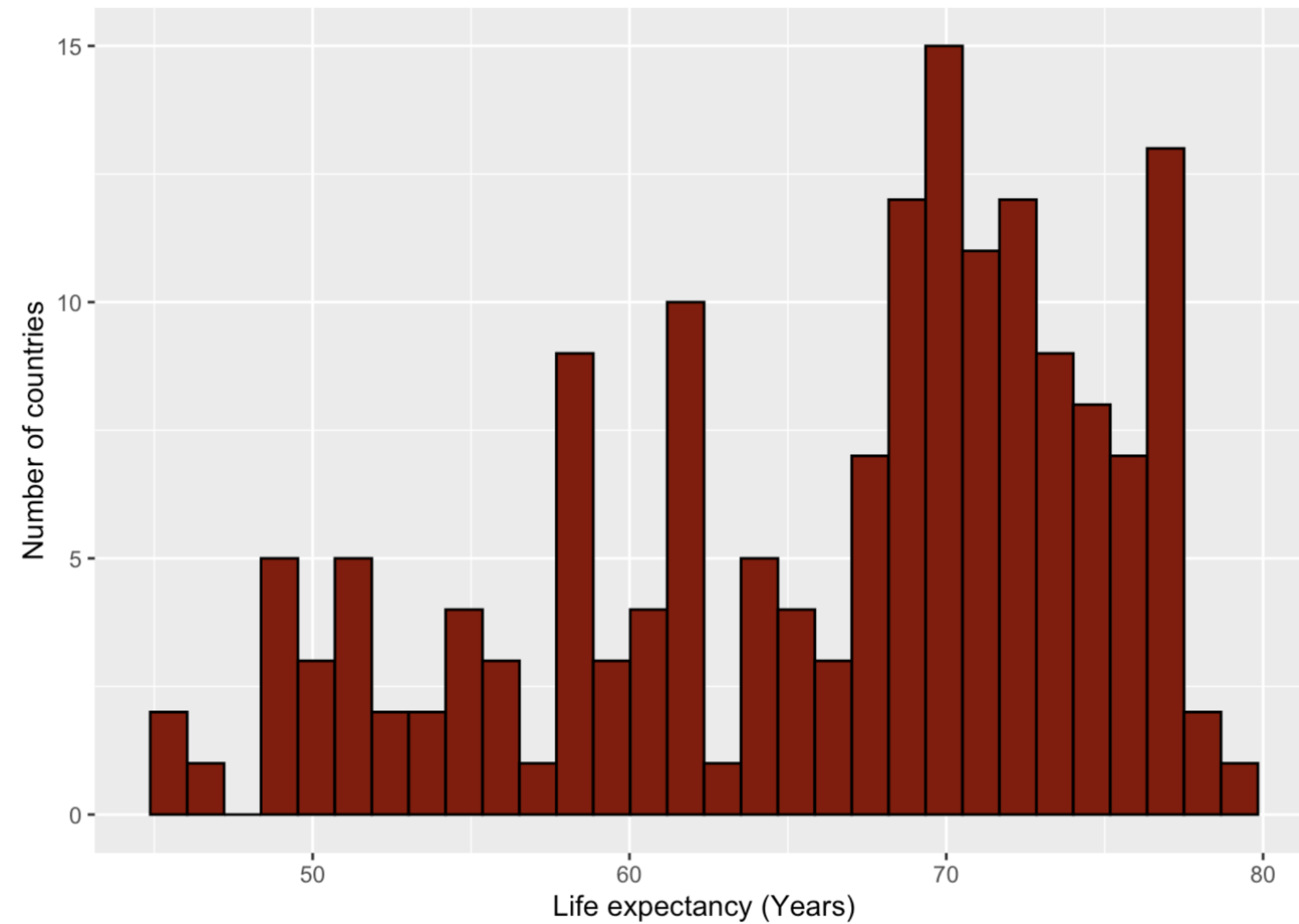
- Need to check that the distribution of each continuous variable is normally distributed – if not, need to report the **median** and **IQR**

```
table_df %>%  
  ggplot(aes(infant_mortality)) +  
  geom_histogram(color = "black", fill = "darkred") +  
  xlab("Infant mortality rate (per 1,000 births)") +  
  ylab("Number of countries")
```

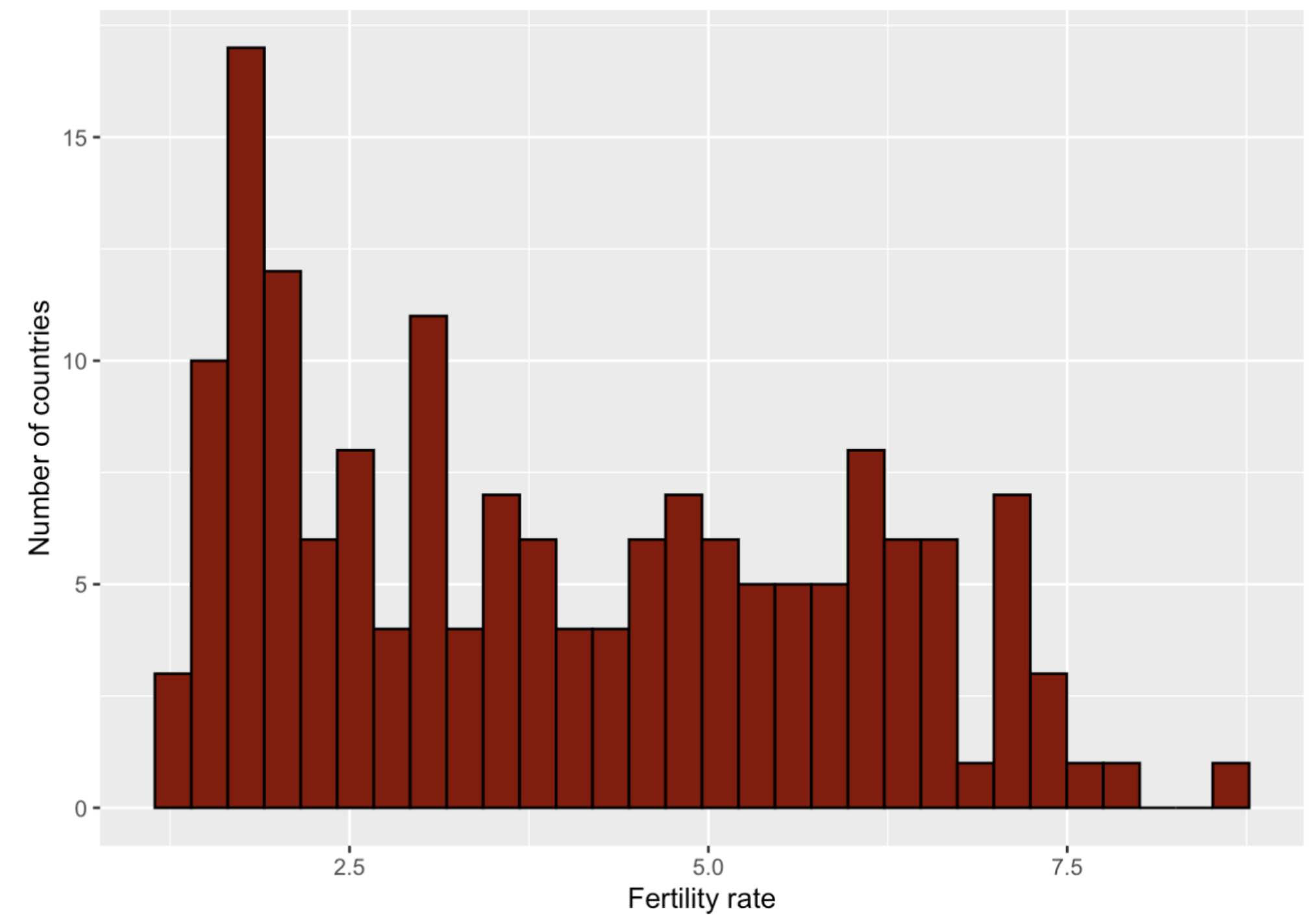


Variable distributions

```
table_df %>%  
  ggplot(aes(life_expectancy)) +  
  geom_histogram(color = "black", fill = "darkred") +  
  xlab("Life expectancy (Years)") +  
  ylab("Number of countries")
```

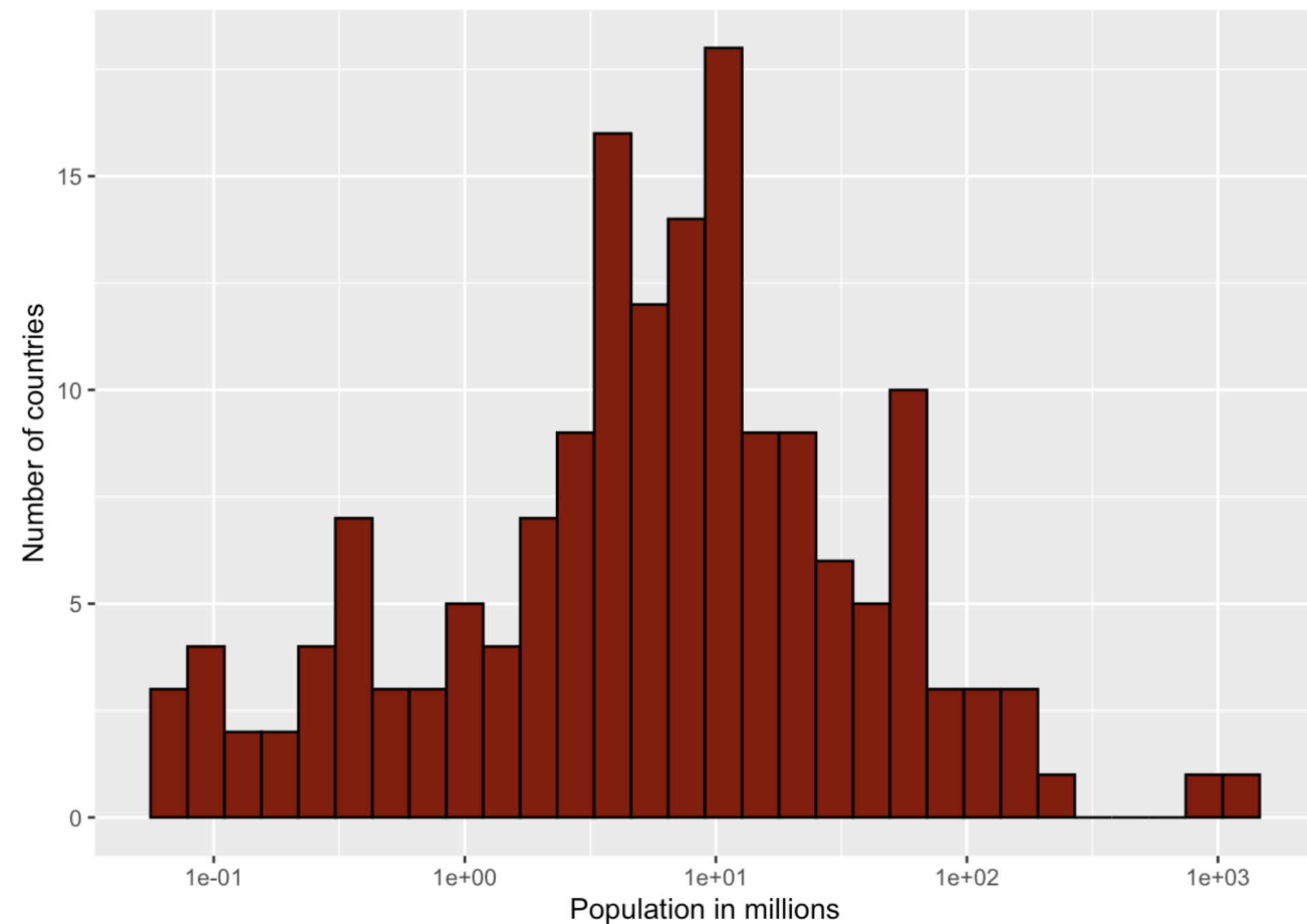


```
table_df %>%  
  ggplot(aes(fertility)) +  
  geom_histogram(color = "black", fill = "darkred") +  
  xlab("Fertility rate") +  
  ylab("Number of countries")
```

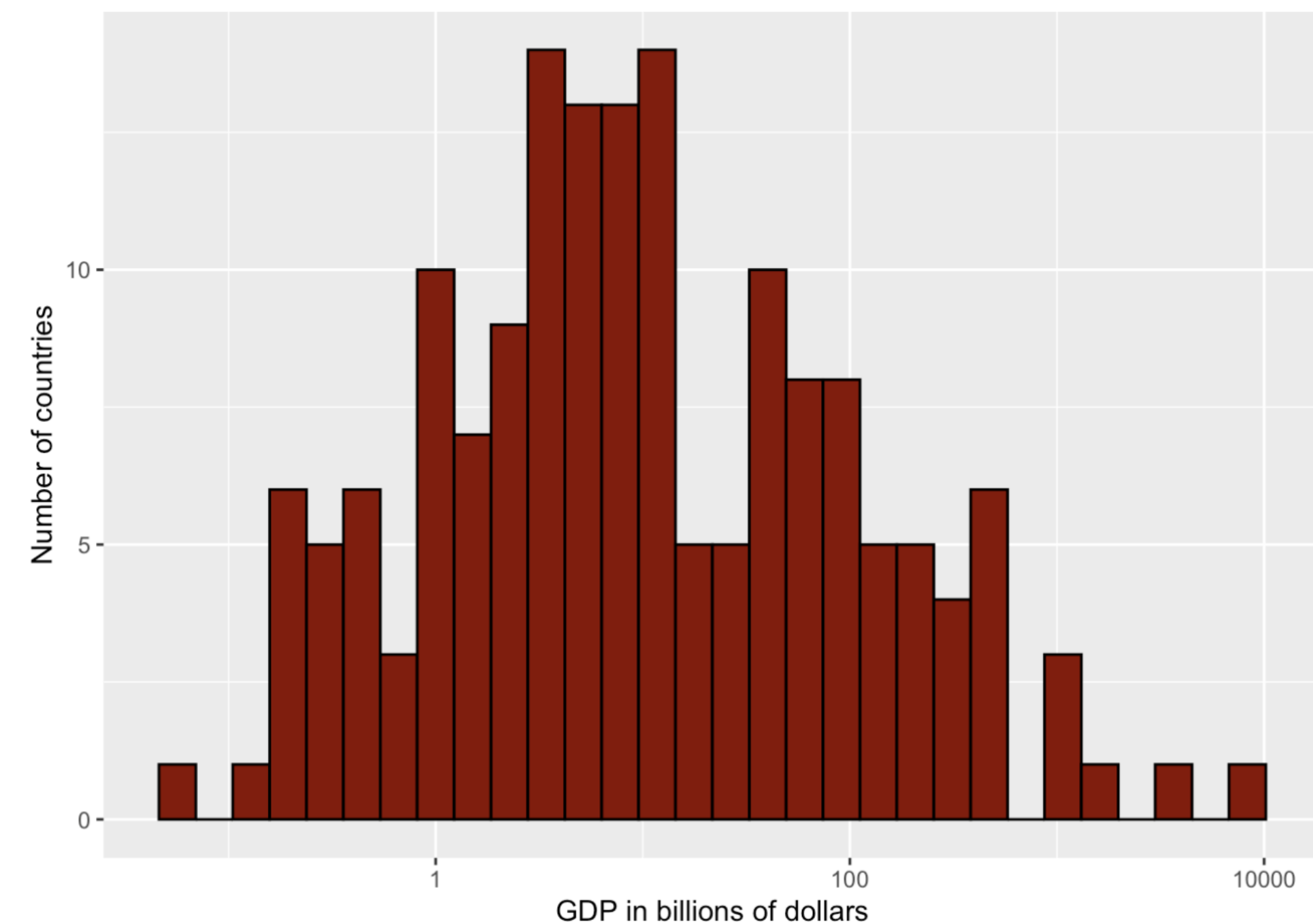


Variable distributions

```
table_df %>%  
  ggplot(aes(population/106)) +  
  geom_histogram(color = "black", fill = "darkred") +  
  xlab("Population in millions") +  
  ylab("Number of countries") +  
  scale_x_log10()
```



```
table_df %>%  
  ggplot(aes(gdp/109)) +  
  geom_histogram(color = "black", fill = "darkred") +  
  xlab("GDP in billions of dollars") +  
  ylab("Number of countries") +  
  scale_x_log10()
```



nonnormal

Define a vector of column names containing all variables that are not normally distributed

- Use the argument **nonnormal** = and the vector created in the **kableone** function

```
nonNormalVars <- c("infant_mortality", "life_expectancy",  
                  "fertility", "population", "gdp")  
  
t1 <- CreateTableOne(vars = vars, data = table_df)  
options(digits = 2)  
kableone(t1, nonnormal = nonNormalVars)
```

	Overall
n	164
infant_mortality (median [IQR])	38.40 [16.00, 77.15]
life_expectancy (median [IQR])	69.15 [60.40, 72.53]
fertility (median [IQR])	3.70 [2.16, 5.59]
population (median [IQR])	6765083.00 [2019378.75, 18343744.75]
gdp (median [IQR])	8224534133.00 [2170758367.50, 51772082085.75]
continent (%)	
Africa	49 (29.9)
Americas	32 (19.5)
Asia	38 (23.2)
Europe	35 (21.3)
Oceania	10 (6.1)

strata

Stratification by a categorical variable is supported

- Use the **strata =** argument and include the categorical variable name in quotes

```
vars <- c("infant_mortality", "life_expectancy",
          "fertility", "population", "gdp")

nonNormalVars <- c("infant_mortality", "life_expectancy",
                  "fertility", "population", "gdp")

t1 <- CreateTableOne(vars = vars, data = table_df,
                    strata = "continent")

options(digits = 2)
kableone(t1, nonnormal = nonNormalVars)
```

	Africa	Americas	Asia	Europe	Oceania	p	test
n	49	32	38	35	10		
infant_mortality (median [IQR])	92.70 [63.00, 119.80]	26.30 [19.45, 45.30]	41.45 [21.35, 74.75]	11.20 [7.35, 16.65]	27.50 [20.22, 40.60]	<0.001	nonnorm
life_expectancy (median [IQR])	56.10 [51.50, 61.30]	71.50 [69.55, 73.35]	68.70 [64.33, 71.72]	74.20 [71.20, 76.75]	64.70 [60.55, 69.10]	<0.001	nonnorm
fertility (median [IQR])	6.21 [5.35, 6.65]	3.09 [2.60, 3.83]	3.80 [2.87, 5.08]	1.81 [1.57, 2.00]	4.72 [3.71, 4.95]	<0.001	nonnorm
population (median [IQR])	7514201.00 [1597534.00, 13371971.00]	4558551.50 [642280.50, 14821391.25]	12206319.00 [3405013.50, 55625548.25]	8821111.00 [4302244.50, 19202250.00]	237357.00 [108906.50, 2730307.00]	0.001	nonnorm
gdp (median [IQR])	2590637104.00 [1019207023.00, 7062040511.00]	7961843305.00 [2664939559.50, 37181495423.25]	13348138779.00 [4889136750.50, 69268340898.75]	71953291930.00 [14464231648.50, 193917649818.00]	267752123.70 [184648934.78, 2101457579.25]	<0.001	nonnorm



strata

Stratification by a categorical variable is supported

- Use the **strata =** argument and include the categorical variable name in quotes
- Can add an overall column back to the table with **addOverall = TRUE**

```
vars <- c("infant_mortality", "life_expectancy",  
          "fertility", "population", "gdp")  
  
nonNormalVars <- c("infant_mortality", "life_expectancy",  
                   "fertility", "population", "gdp")  
  
t1 <- CreateTableOne(vars = vars, data = table_df,  
                     strata = "continent", addOverall = TRUE)  
  
options(digits = 2)  
kableone(t1, nonnormal = nonNormalVars)
```

	Overall	Africa	Americas	Asia	Europe	Oceania	p	test
n	164	49	32	38	35	10		
infant_mortality (median [IQR])	38.40 [16.00, 77.15]	92.70 [63.00, 119.80]	26.30 [19.45, 45.30]	41.45 [21.35, 74.75]	11.20 [7.35, 16.65]	27.50 [20.22, 40.60]	<0.001	nonnorm
life_expectancy (median [IQR])	69.15 [60.40, 72.53]	56.10 [51.50, 61.30]	71.50 [69.55, 73.35]	68.70 [64.33, 71.72]	74.20 [71.20, 76.75]	64.70 [60.55, 69.10]	<0.001	nonnorm
fertility (median [IQR])	3.70 [2.16, 5.59]	6.21 [5.35, 6.65]	3.09 [2.60, 3.83]	3.80 [2.87, 5.08]	1.81 [1.57, 2.00]	4.72 [3.71, 4.95]	<0.001	nonnorm
population (median [IQR])	6765083.00 [2019378.75, 18343744.75]	7514201.00 [1597534.00, 13371971.00]	4558551.50 [642280.50, 14821391.25]	12206319.00 [3405013.50, 55625548.25]	8821111.00 [4302244.50, 19202250.00]	237357.00 [108906.50, 2730307.00]	0.001	nonnorm
gdp (median [IQR])	8224534133.00 [2170758367.50, 51772082085.75]	2590637104.00 [1019207023.00, 7062040511.00]	7961843305.00 [2664939559.50, 37181495423.25]	13348138779.00 [4889136750.50, 69268340898.75]	71953291930.00 [14464231648.50, 193917649818.00]	267752123.70 [184648934.78, 2101457579.25]	<0.001	nonnorm



Finishing touches

- Convert population and GDP values
- Rename variables

```
table_df <- gapminder %>%  
  filter(year == 1990) %>%  
  mutate(Population = population/10^6,  
         GDP = gdp/10^9) %>%  
  rename("Infant mortality" = infant_mortality,  
         "Life expectancy" = life_expectancy,  
         "Fertility" = fertility)  
  
table_df <- table_df[complete.cases(table_df),]  
  
vars <- c("Infant mortality", "Life expectancy",  
         "Fertility", "Population", "GDP")  
  
nonNormalVars <- c("Infant mortality", "Life expectancy",  
                  "Fertility", "Population", "GDP")  
  
t1 <- CreateTableOne(vars = vars, data = table_df,  
                   strata = "continent", addOverall = TRUE)  
  
options(digits = 2)  
kableone(t1, nonnormal = nonNormalVars)
```

Finishing touches

	Overall	Africa	Americas	Asia	Europe	Oceania	p	test
n	164	49	32	38	35	10		
Infant mortality (median [IQR])	38.40 [16.00, 77.15]	92.70 [63.00, 119.80]	26.30 [19.45, 45.30]	41.45 [21.35, 74.75]	11.20 [7.35, 16.65]	27.50 [20.22, 40.60]	<0.001	nonnorm
Life expectancy (median [IQR])	69.15 [60.40, 72.53]	56.10 [51.50, 61.30]	71.50 [69.55, 73.35]	68.70 [64.33, 71.72]	74.20 [71.20, 76.75]	64.70 [60.55, 69.10]	<0.001	nonnorm
Fertility (median [IQR])	3.70 [2.16, 5.59]	6.21 [5.35, 6.65]	3.09 [2.60, 3.83]	3.80 [2.87, 5.08]	1.81 [1.57, 2.00]	4.72 [3.71, 4.95]	<0.001	nonnorm
Population (median [IQR])	6.77 [2.02, 18.34]	7.51 [1.60, 13.37]	4.56 [0.64, 14.82]	12.21 [3.41, 55.63]	8.82 [4.30, 19.20]	0.24 [0.11, 2.73]	0.001	nonnorm
GDP (median [IQR])	8.22 [2.17, 51.77]	2.59 [1.02, 7.06]	7.96 [2.66, 37.18]	13.35 [4.89, 69.27]	71.95 [14.46, 193.92]	0.27 [0.18, 2.10]	<0.001	nonnorm



Summary

- In scientific journal articles, data is usually summarized in one table, referred to as “Table 1”
- The **tableone** package facilitates the creation of a “Table 1”
- The mean and standard deviation, or the median and IQR can be reported for continuous variables

Summarizing data

Reporting missing data

Heather Mattie, PhD

Missing data

Missing data is a common issue in biomedical datasets

- Can bias results and reduce statistical power
- Some models and functions will not run if there is missing data
- Clear communication of missingness improves reproducibility and transparency



gapminder example

```
library(tidyverse)
library(dslabs)
data(gapminder)

df <- gapminder %>%
  filter(year == 1965)

head(df)
```

##	country	year	infant_mortality	life_expectancy	fertility
## 1	Albania	1965	94.10	66.59	5.59
## 2	Algeria	1965	149.10	50.09	7.66
## 3	Angola	1965	NA	38.74	7.43
## 4	Antigua and Barbuda	1965	NA	65.23	4.19
## 5	Argentina	1965	59.11	65.95	3.06
## 6	Armenia	1965	NA	69.26	3.98
##	population	gdp	continent	region	
## 1	1896125	NA	Europe	Southern Europe	
## 2	12626953	14486635612	Africa	Northern Africa	
## 3	5765025	NA	Africa	Middle Africa	
## 4	59653	NA	Americas	Caribbean	
## 5	22283389	130560392722	Americas	South America	
## 6	2204650	NA	Asia	Western Asia	



is.na

The **is.na** function returns TRUE if a value is NA (missing) and FALSE if it is not NA

- Using **is.na** with the **sum** function shows how many values are NAs in an object

```
sum(is.na(df))
```

```
## [1] 149
```

```
sum(is.na(df$country))
```

```
## [1] 0
```

```
sum(is.na(df$year))
```

```
## [1] 0
```

```
sum(is.na(df$infant_mortality))
```

```
## [1] 66
```

```
sum(is.na(df$life_expectancy))
```

```
## [1] 0
```

```
sum(is.na(df$fertility))
```

```
## [1] 0
```

```
sum(is.na(df$population))
```

```
## [1] 0
```

```
sum(is.na(df$gdp))
```

```
## [1] 83
```

```
sum(is.na(df$continent))
```

```
## [1] 0
```

```
sum(is.na(df$region))
```

```
## [1] 0
```

colSums

The **colSums** function with **is.na** returns the number of NAs in each column

```
colSums(is.na(df))
```

```
##          country          year infant_mortality life_expectancy
##              0              0              66              0
##    fertility    population          gdp          continent
##              0              0              83              0
##          region
##              0
```

rowSums

The **rowSums** function with **is.na** returns the number of NAs in each row

```
rowSums(is.na(df))
```

```
##      [1] 1 0 2 2 0 2 2 0 0 2 1 1 0 0 2 0 1 0 2 0 2 0 0 2 1 0 0 2 0 0 2 0 1 0 0 0 2
##     [38] 1 0 0 0 2 1 2 2 0 2 0 0 0 0 2 2 2 2 0 0 0 1 1 1 1 2 0 0 2 2 0 1 2 0 1 0 1
##     [75] 0 0 0 0 0 1 1 1 0 1 0 1 2 0 1 0 1 2 2 1 1 0 0 1 2 0 2 2 0 0 0 1 1 1 0 1 0
##    [112] 2 2 2 2 0 1 2 0 0 1 1 0 1 0 0 0 0 0 0 0 0 1 0 1 2 2 2 0 2 1 2 2 0 2 0 0
##    [149] 0 2 2 1 1 0 0 0 2 1 0 1 0 2 1 0 2 0 1 0 0 0 2 1 2 1 0 0 0 2 1 0 2 1 1 0 0
```

skimr and skim

The **skimr** package provides compact and flexible ways to generate summary statistics

- The **skim** function outputs a comprehensive data summary

```
library(skimr)  
skim(df)
```

Data summary

Name	df
Number of rows	185
Number of columns	9

Column type frequency:

factor	3
numeric	6

Group variables	None
-----------------	------

skimr and skim

Variable type: factor

skim_variable	n_missing	complete_rate	ordered	n_unique	top_counts
country	0	1	FALSE	185	Alb: 1, Alg: 1, Ang: 1, Ant: 1
continent	0	1	FALSE	5	Afr: 51, Asi: 47, Eur: 39, Ame: 36
region	0	1	FALSE	22	Wes: 18, Eas: 16, Wes: 16, Car: 13

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
year	0	1.00	1.965000e+03	0.000000e+00	1965.00	1.96500e+03	1.965000e+03	1.965000e+03	1.965000e+03	
infant_mortality	66	0.64	9.711000e+01	5.817000e+01	13.70	5.15500e+01	9.620000e+01	1.372000e+02	2.636000e+02	
life_expectancy	0	1.00	5.726000e+01	1.109000e+01	31.26	4.84400e+01	5.834000e+01	6.705000e+01	7.400000e+01	
fertility	0	1.00	5.380000e+00	1.830000e+00	1.81	3.60000e+00	6.070000e+00	6.800000e+00	8.200000e+00	
population	0	1.00	1.759520e+07	6.650895e+07	37963.00	7.53285e+05	3.559687e+06	9.504702e+06	7.065909e+08	
gdp	83	0.55	7.915212e+10	3.387851e+11	110094728.00	1.41424e+09	4.138937e+09	2.746349e+10	3.162743e+12	



naniar

The **naniar** package is a **tidyverse**-friendly package designed to facilitate the exploration, visualization, and manipulation of missing data.

- It provides a suite of tools for understanding the patterns and mechanisms of missingness within a dataset



gg_miss_var

The **gg_miss_var** function is used to visualize the missing data in a dataset by variable.

- Generates a **ggplot2** plot that displays the number of missing values for each variable (column) in a data frame

```
library(naniar)

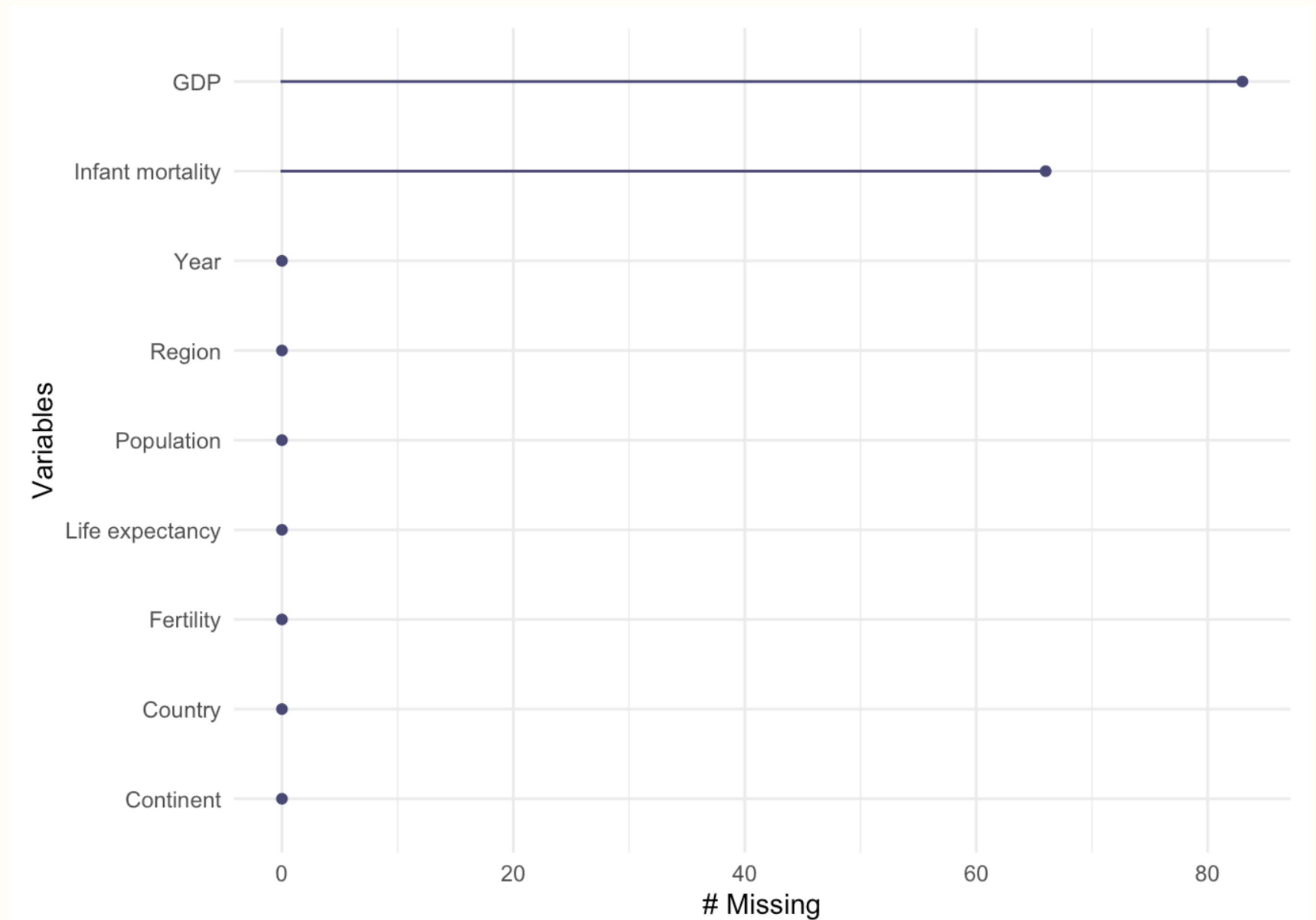
df <- df %>%
  rename("Infant mortality" = infant_mortality,
         "Life expectancy" = life_expectancy,
         "Population" = population,
         "Fertility" = fertility,
         "Continent" = continent,
         "Country" = country,
         "Region" = region,
         "Year" = year,
         "GDP" = gdp
  )

gg_miss_var(df)
```

gg_miss_var

The **gg_miss_var** function is used to visualize the missing data in a dataset by variable.

- Generates a **ggplot2** plot that displays the number of missing values for each variable (column) in a data frame

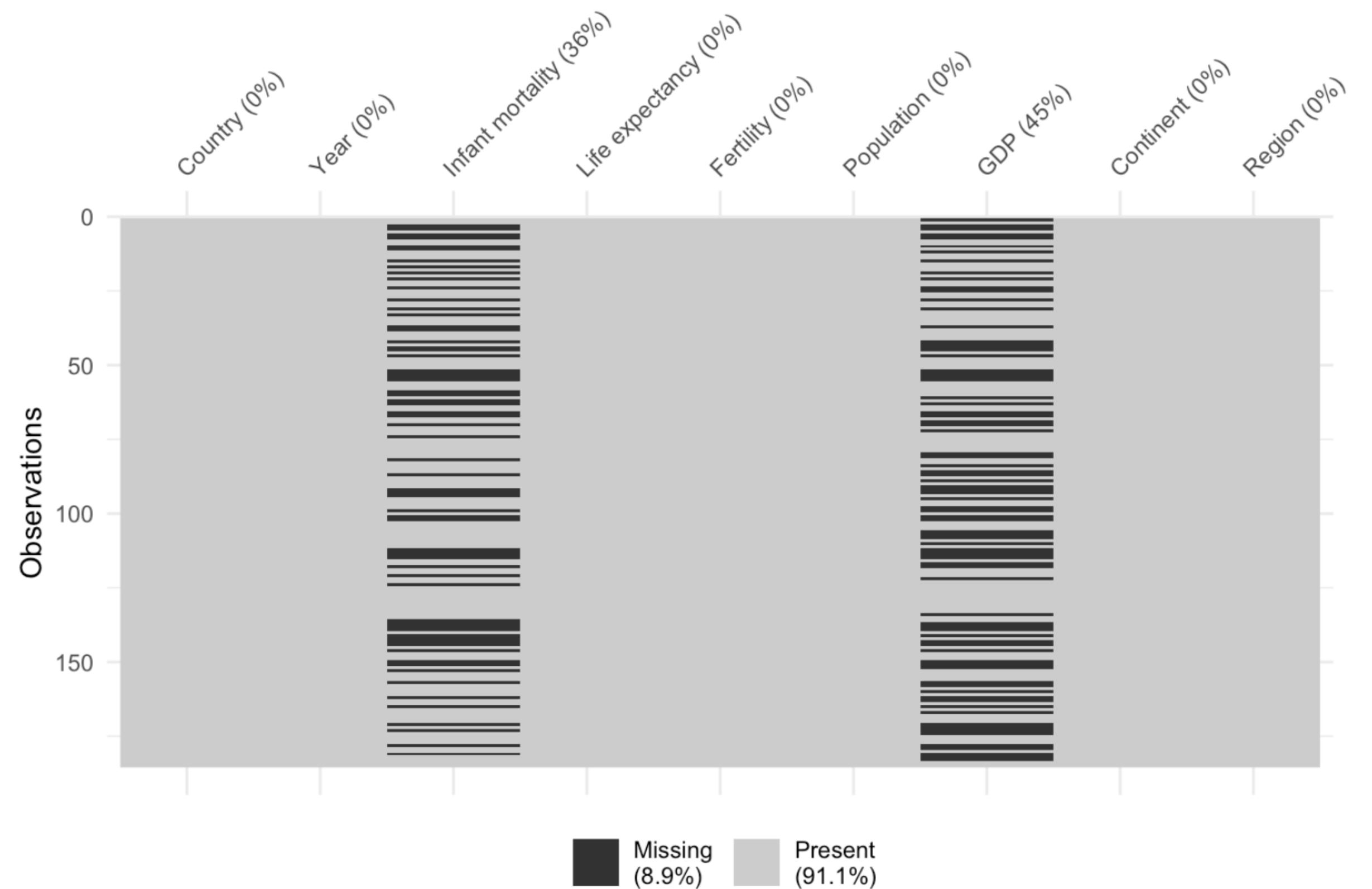


vis_miss

The **vis_miss** function allows for the visualization of missing data patterns within a data frame.

- Generates a **ggplot** object, offering a quick and visual overview of where missing values and present values are located within the dataset.

```
vis_miss(df)
```



Summary

- Missing data is a common issue and should be reported
- The **colSums** and **rowSums** can be used to show the number of missing values in columns and rows
- The **skim** function can be used to generate a comprehensive missing data summary
- Missing data can be visualized by using the **gg_miss_var** and **vis_miss** functions